คู่มืออ้างอิงคำสั่ง Git สำหรับ DevOps

คู่มือฉบับนี้รวบรวมคำสั่ง Git ที่สำคัญ แบ่งตามสถานการณ์การทำงานจริง เช่น การอัปโหลดโค้ด การดาวน์โหลดโค้ด การแก้ไฟล์ ย้อนโค้ด สร้าง Branch และการทำงานร่วมกับทีม

คำศัพท์พื้นฐานที่ควรทราบ

คำศัพท์	ความหมาย
Local	Git Repository ในคอมพิวเตอร์ของเรา
Remote	Repository บน GitHub หรือเซิร์ฟเวอร์
Commit	บันทึกประวัติการเปลี่ยนแปลง
Branch	แขนงการพัฒนาโค้ด (แยกส่วนการทำงาน)
Repository (Repo)	พื้นที่เก็บโปรเจกต์และประวัติโค้ด

1. คำสั่งเริ่มต้นและตั้งค่า Git

ใช้เมื่อติดตั้ง Git ใหม่ หรือเริ่มต้นโปรเจกต์ใหม่

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git --version`	ตรวจสอบเวอร์ชัน Git
`git config --global user.name "ชื่อ"`	ตั้งชื่อผู้ทำ Commit
`git config --global user.email "อีเมล"`	ตั้งอีเมลผู้ทำ Commit
`git config --list`	ดูการตั้งค่าทั้งหมด
`git config --global --list`	ดูการตั้งค่าระดับผู้ใช้
`git config --global --edit`	เปิดแก้ไขไฟล์ตั้งค่า
`git config --global init.defaultBranch main`	ตั้ง Branch เริ่มต้นเป็น main
`git config --global core.editor "code --wait"`	ตั้ง VS Code เป็น Editor
`git config --unset ชื่อค่า`	ลบค่าที่กำหนด

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git config --global --unset ชื่อค่า`	ลบค่าระดับ Global

2. คำสั่งสร้างและดาวน์โหลดโปรเจกต์

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git init`	เริ่มต้น Git ในโฟลเดอร์ปัจจุบัน
`git init ชื่อโฟลเดอร์`	สร้าง Repository ในโฟลเดอร์ใหม่
`git clone URL`	ดาวน์โหลดโปรเจกต์จาก GitHub
`git clone URL ชื่อโฟลเดอร์`	ดาวน์โหลดและกำหนดชื่อโฟลเดอร์
`git clone --depth 1 URL`	ดาวน์โหลดเฉพาะประวัติแบบตื้น เพื่อลดข้อมูล
`git clone --branch ชื่อ URL`	ดาวน์โหลด Branch ที่ระบุ
`git clone --single-branch URL`	ดาวน์โหลดเฉพาะ Branch ที่เลือก
`git init --bare`	สร้าง Repository เปล่าสำหรับเป็น Remote Server

3. คำสั่งตรวจสอบสถานะและประวัติ

ใช้เมื่อต้องการตรวจสอบว่าไฟล์ไหนเปลี่ยนแปลงไปแล้ว หรือใครแก้ไขโค้ด

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git status`	ตรวจสอบสถานะไฟล์
`git status -s`	ดูสถานะแบบสั้น
`git log`	ดูประวัติ Commit
`git log --oneline`	ดูประวัติแบบบรรทัดเดียว
`git log --graph`	ดูโครงสร้างประวัติเป็นกราฟ
`git log --all --oneline --graph`	ดูประวัติทุก Branch
`git log -5`	ดู 5 Commit ล่าสุด
`git log --author="ชื่อ"`	ดู Commit ของผู้เขียน
`git log --since="7 days ago"`	ดู Commit ในช่วง 7 วันที่ผ่านมา

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git show [รหัสCommit]`	ดูรายละเอียด Commit ล่าสุด หรือที่ระบุรหัส
`git diff`	ดูความแตกต่างของไฟล์ที่ยังไม่ Stage
`git diff --staged`	ดูความแตกต่างของไฟล์ที่ Stage แล้ว
`git blame ชื่อไฟล์`	ดูว่าใครแก้ไขแต่ละบรรทัด
`git reflog`	ดูประวัติการเปลี่ยนตำแหน่ง HEAD

4. คำสั่ง Add และ Commit

ส่วนนี้เป็นหัวใจหลักของ Git ใช้เมื่อแก้ไขไฟล์และต้องการบันทึกงาน

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git add .`	เพิ่มไฟล์ที่เปลี่ยนแปลงในโฟลเดอร์ปัจจุบันและโฟลเดอร์ย่อย
`git add -A`	เพิ่มการเปลี่ยนแปลงทั้งหมดใน Repository
`git add ชื่อไฟล์`	เพิ่มไฟล์ที่ต้องการ
`git add *.txt`	เพิ่มไฟล์ที่ตรงกับรูปแบบที่กำหนด
`git add -p`	เลือกเพิ่มเฉพาะบางส่วนของการแก้ไข
`git commit -m "ข้อความ"`	บันทึก Commit พร้อมข้อความ
`git commit`	เปิด Editor เพื่อเขียนข้อความ Commit
`git commit -a -m "ข้อความ"`	Stage และ Commit ไฟล์ที่ Git ติดตามอยู่แล้ว
`git commit --amend`	แก้ไข Commit ล่าสุด
`git commit --amend -m "ข้อความ"`	เปลี่ยนข้อความ Commit ล่าสุด
`git commit --allow-empty -m "..."`	สร้าง Commit ที่ไม่มีการเปลี่ยนแปลงไฟล์

> **หมายเหตุ:** `git add .` กับ `git add -A` มีขอบเขตต่างกัน โดยคำสั่งหลังครอบคลุมการเปลี่ยนแปลงทั้ง Repository

5. คำสั่งอัปโหลด Git ขึ้น GitHub (Push)

ใช้เมื่อแก้ไขโค้ดเสร็จแล้ว ต้องการส่งงานจากคอมพิวเตอร์ขึ้น GitHub (ส่ง Commit ไม่ใช่ไฟล์ที่ยังไม่ Commit)

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git push`	ส่ง Commit ขึ้น Remote
`git push origin main`	ส่ง Branch main ขึ้น GitHub
`git push -u origin main`	Push พร้อมตั้งค่าการติดตาม Branch
`git push -u origin ชื่อBranch`	ส่ง Branch ใหม่และตั้ง Upstream
`git push --all origin`	ส่ง Branch ในเครื่องทั้งหมด
`git push origin --tags`	ส่ง Tag ทั้งหมด
`git push --force-with-lease`	Force Push โดยตรวจสอบว่า Remote ไม่ได้เปลี่ยนจากที่เรารู้
`git push --delete origin ชื่อBranch`	ลบ Branch บน GitHub

ตัวอย่างการอัปโหลดครั้งแรก

```
git remote add origin git@github.com:USERNAME/project.git
git branch -M main
git push -u origin main
```

6. คำสั่งดาวน์โหลดและอัปเดต (Pull / Fetch)

ใช้เมื่อมีคนอื่นแก้ไขโปรเจกต์บน GitHub แล้วเราต้องการอัปเดตโค้ดในเครื่อง

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git fetch`	ดึงข้อมูลล่าสุดจาก Remote แต่ยังไม่รวมเข้า Branch ปัจจุบัน
`git fetch --prune`	ดึงข้อมูลพร้อมล้าง Remote Branch ที่ถูกลบไปแล้ว
`git pull`	ดึงและรวมงานล่าสุดเข้ากับ Branch ปัจจุบัน
`git pull origin main`	ดึง Branch main จาก GitHub
`git pull --rebase`	ดึงงานและ Rebase งานของเราให้อยู่ต่อจากงานล่าสุด
`git pull --autostash`	พักการแก้ไขที่ยังไม่ Commit ชั่วคราวระหว่าง Pull

> **ข้อแตกต่าง:** `Fetch` = ดึงข้อมูลมาเช็กก่อน | `Pull` = ดึงข้อมูลและรวมเข้ากับงานเราทันที

7. คำสั่งจัดการ Remote Repository

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git remote -v`	ดูชื่อและ URL ของ Remote ทั้งหมด
`git remote add origin URL`	เชื่อม GitHub เข้ากับโปรเจกต์
`git remote remove origin`	ยกเลิก Remote
`git remote set-url origin URL`	เปลี่ยน URL ของ Remote (เช่น เปลี่ยน HTTPS เป็น SSH)
`git ls-remote origin`	ดู Branch และ Tag ที่อยู่บน Remote

8. คำสั่งจัดการ Branch

Branch ใช้แยกงานแต่ละส่วนออกจากกัน เพื่อไม่ให้การพัฒนาฟีเจอร์ใหม่ไปรบกวนโค้ดหลัก

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git branch`	ดู Branch ในเครื่อง (เพิ่ม -a เพื่อดู Remote ด้วย)
`git branch ชื่อ`	สร้าง Branch ใหม่
`git branch -d ชื่อ`	ลบ Branch ที่ Merge แล้ว (-D เพื่อบังคับลบ)
`git switch main`	ย้ายไป Branch main
`git switch -c ชื่อ`	สร้างและย้ายไป Branch ใหม่ (แทน git checkout -b)
`git switch -`	กลับไป Branch ก่อนหน้า
`git merge ชื่อ`	รวม Branch ระบุเข้ากับ Branch ปัจจุบัน
`git merge --abort`	ยกเลิก Merge ที่ยังไม่เสร็จ

9. คำสั่งแก้ไขไฟล์ ย้อนกลับ และลบไฟล์

ใช้เมื่อแก้โค้ดผิด ลบไฟล์ผิด หรือเพิ่มไฟล์ผิด

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git restore ชื่อไฟล์`	ยกเลิกการแก้ไขไฟล์ที่ยังไม่ Stage (ใช้ . เพื่อคืนค่าทั้งหมด)
`git restore --staged ชื่อไฟล์`	เอาไฟล์ออกจาก Staging แต่เก็บการแก้ไขไว้
`git rm --cached ชื่อไฟล์`	หยุดติดตามไฟล์ แต่เก็บไฟล์ไว้ในเครื่อง
`git reset --soft HEAD~1`	ย้อน Commit แต่เก็บงานใน Staging
`git reset --hard HEAD~1`	ย้อน Commit และ **ทิ้งการแก้ไข** ที่เกี่ยวข้อง
`git revert HEAD`	สร้าง Commit ใหม่เพื่อย้อนผลของ Commit ล่าสุด
`git clean -fd`	ลบไฟล์และโฟลเดอร์ Untracked (-n เพื่อตรวจสอบก่อนลบ)

> **คำสั่งอันตราย:** ระวัง `git reset --hard`, `git clean -fd` และ `git push --force` อาจทำให้ข้อมูลหาย หรือเขียนทับประวัติของทีมได้ ให้ตรวจสอบ `git status` ก่อนเสมอ

10. คำสั่งจัดการ Commit และประวัติ

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git commit --amend --no-edit`	เพิ่มการเปลี่ยนแปลงเข้า Commit ล่าสุดโดยไม่เปลี่ยนข้อความ
`git cherry-pick รหัสCommit`	เลือก Commit จาก Branch อื่นมาใช้
`git rebase main`	นำ Commit ของ Branch ปัจจุบันไปต่อจาก Main
`git rebase -i HEAD~3`	จัดการ Commit ย้อนหลัง 3 รายการ (แก้ไข/รวม)

11. คำสั่ง Stash (พักงานชั่วคราว)

ใช้เมื่อกำลังทำงานหนึ่งอยู่ แต่ต้องสลับ Branch ไปแก้งานอื่นด่วน

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git stash`	เก็บงานที่แก้ไขชั่วคราว
`git stash push -m "ข้อความ"`	เก็บงานพร้อมตั้งชื่อ
`git stash list`	ดูรายการงานที่พักไว้
`git stash pop`	นำงานล่าสุดกลับมาและนำรายการ Stash ออก
`git stash apply`	นำงานล่าสุดกลับมา แต่ยังเก็บ Stash ไว้
`git stash clear`	ลบ Stash ทั้งหมด

12. คำสั่ง Merge Conflict

ปัญหาที่เกิดเมื่อ Git รวมการแก้ไขจากหลายคนอัตโนมัติไม่ได้ วิธีแก้คือ เปิดไฟล์ เลือกโค้ดที่ถูกต้อง ลบเครื่องหมาย `<<<<` `====` `>>>>` ออก แล้ว `git add`

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git diff`	ตรวจสอบส่วนที่ขัดแย้ง
`git merge --continue`	ดำเนินการ Merge ต่อหลังจากแก้ไฟล์และ add แล้ว
`git merge --abort`	ยกเลิก Merge กลับไปสถานะก่อนหน้า

13. .gitignore และจัดการไฟล์ที่ไม่ต้องการติดตาม

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git check-ignore -v ชื่อไฟล์`	ดูว่ากฎจากไฟล์ไหนเป็นตัว Ignore ไฟล์นี้
`git rm --cached .env`	หยุดติดตามไฟล์ .env แต่ไม่ลบไฟล์จริง

> **ข้อควรระวัง:** ถ้าไฟล์ .env มี Password ที่เคย Push ขึ้น GitHub แล้ว การเพิ่ม .gitignore ไม่ทำให้ข้อมูลเดิมบนเว็บหายไป ต้องทำการเปลี่ยน Password นั้นทันที

14. คำสั่ง SSH Key สำหรับ GitHub

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`ssh-keygen -t ed25519 -C "email"`	สร้าง SSH Key คู่ใหม่
`eval "$(ssh-agent -s)"`	เปิด SSH Agent ใน Git Bash
`ssh-add ~/.ssh/id_ed25519`	เพิ่ม Private Key เข้า Agent
`ssh -T git@github.com`	ทดสอบเชื่อมต่อ GitHub

15. คำสั่ง Tag และ Version

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git tag -a v1.0.0 -m "Release"`	สร้าง Annotated Tag พร้อมคำอธิบาย
`git push origin v1.0.0`	ส่ง Tag ขึ้น GitHub (หรือ `--tags` เพื่อส่งทั้งหมด)

16. คำสั่งค้นหาและตรวจสอบโค้ด

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git grep -n "ข้อความ"`	ค้นหาข้อความในไฟล์ที่ Git ติดตาม พร้อมเลขบรรทัด
`git log -S"ข้อความ"`	ค้นหา Commit ที่เพิ่มหรือลบข้อความนั้น
`git diff --name-only`	ดูเฉพาะชื่อไฟล์ที่เปลี่ยน

17. คำสั่งขั้นสูงสำหรับ DevOps (Advanced)

คำสั่ง	ใช้ตอนไหน / ทำอะไร
`git bisect start / bad / good`	ค้นหา Commit ที่ทำให้เกิด Bug แบบ Binary Search
`git worktree add [path] [branch]`	สร้างโฟลเดอร์ทำงานอีกชุด แยก Branch โดยไม่ต้อง Clone ใหม่
`git submodule update --init`	ดาวน์โหลดและอัปเดต Submodule
`git archive -o project.zip HEAD`	สร้างไฟล์ ZIP จาก Commit ปัจจุบัน
`git reflog`	ค้นหาประวัติการเปลี่ยนตำแหน่ง HEAD (กู้คืน Commit ที่หายไปได้)

18. ตัวอย่าง Workflow ตามงานที่ได้รับมอบหมาย

กรณีที่ 1: ต้องการดาวน์โหลดและอัปเดตโค้ดล่าสุดมาแก้ไข

```
git switch main
git pull origin main
git switch -c feature-new  # แยก Branch สำหรับทำงานใหม่
```

กรณีที่ 2: แก้ไขงานเสร็จ และต้องการอัปโหลด

```
git status
git add .
git commit -m "Add new feature"
git push -u origin feature-new
```

กรณีที่ 3: ต้องการส่งงาน แต่มีคนอื่นอัปเดตโค้ด Main ไปก่อนเรา

```
git fetch origin
git merge origin/main
# (หากมี Conflict ให้เข้าไปแก้ไฟล์ -> git add . -> git commit)
git push
```

กรณีที่ 4: Commit ผิดไปแล้ว แต่ยังไม่ได้ Push

```
git reset --soft HEAD~1
# (แก้ไขไฟล์ที่ต้องการ)
git add .
git commit -m "Corrected message"
```

กรณีที่ 5: Push งานผิดขึ้น GitHub ไปแล้ว

```
git revert HEAD
git push
# วิธีนี้จะสร้าง Commit ใหม่เพื่อย้อนการเปลี่ยนแปลง โดยไม่ลบประวัติเดิม
```

19. สรุปคำสั่งที่ต้องจำสำหรับ DevOps

แนะนำให้แบ่งการจำเป็น 3 ระดับ เพื่อให้ง่ายต่อการนำไปใช้งานจริง:

ระดับ 1: ต้องจำให้ได้ (ใช้ทุกวัน)

- `git status`
- `git add .`
- `git commit -m`
- `git push / pull`
- `git clone`
- `git branch / switch`
- `git merge`
- `git log`

ระดับ 2: ต้องใช้เป็น (แก้ปัญหา)

- `git diff`
- `git fetch`
- `git stash / pop`
- `git restore`
- `git reset / revert`
- `git rebase`
- `git cherry-pick`
- `git reflog`

ระดับ 3: รู้จักไว้ (ขั้นสูง)

- `git bisect`
- `git worktree`
- `git submodule`
- `git tag`
- `git gc / fsck`
- `git bundle`